

Curso online: **Programación orientada a objetos: Java avanzado**

Expresiones Lambda

Autores

Jorge Molinero Muñoz

Miguel Sallent Sánchez

ÍNDICE

1.	Expresiones Lambda.....	3
2.	Los Streams y las expresiones Lambda	5
3.	Ejercicio resuelto	8

1. Expresiones Lambda

Puede que en alguna ocasión nos hayamos encontrado con una expresión de este tipo:

```
List<Persona> listaPersonas = new ArrayList<Persona>();
listaPersonas.forEach(p -> System.out.println(p.getNombre()));
```

Y que no hayamos entendido bien la construcción de este tipo de expresiones, o que no sepamos manejarlas con soltura. Pues bien, la expresión resaltada en negrita se denomina *expresión lambda*.

Una expresión lambda es un pequeño fragmento de código que toma unos valores de entrada y devuelve un valor o realiza una acción. Son por tanto muy similares a los métodos, pero no necesitan tener un nombre y se pueden insertar dentro de un método. Son funciones anónimas, que no necesitan estar vinculadas a una clase.

Para representarlas, se utiliza el símbolo “->”, que separa los parámetros a la izquierda y la expresión a la derecha. Por ejemplo:

parámetro -> expresión

O si hay más de un parámetro, se ponen entre paréntesis:

(parámetro1, parámetro2) -> expresión

Su utilidad es que nos permiten implementar interfaces que tienen un solo método abstracto, ahorrando código. Este tipo de interfaces, que únicamente tienen un método abstracto (ojo, pueden tener más métodos default, es decir con implementación), se denominan **interfaces funcionales**.

Ejemplo 1

Supongamos que tenemos una clase Persona, con atributos nombre, apellido1 y apellido2, todos de tipo String. Tenemos una lista de objetos Persona, y queremos mostrar por pantalla cierto atributo de cada objeto.

Primera solución: utilizar un bucle.

La primera solución que se nos ocurre es utilizar un bucle:

```
List<Persona> listaPersonas = new ArrayList<Persona>();

listaPersonas.add(new Persona("JUAN", "FERNANDEZ", "RODRIGUEZ"));
listaPersonas.add(new Persona("ROSA", "LOPEZ", "ALVAREZ"));
listaPersonas.add(new Persona("MARTIN", "MARQUEZ", "MANRIQUE"));
listaPersonas.add(new Persona("ELENA", "FERNANDEZ", "MORA"));
listaPersonas.add(new Persona("FRANCISCO", "MARTINEZ", "OLIVAR"));
listaPersonas.add(new Persona("MIGUEL", "SANCHEZ", "HERRERA"));

for(Persona p : listaPersonas)
{
    System.out.println(p.getNombre());
}
```

Segunda solución: método foreach.

Ahora observamos que el interfaz [List](#) tiene un método [foreach](#), que toma como argumento un objeto de tipo [Consumer](#), que es un interfaz funcional. Este interfaz tiene un único método abstracto, `accept`, que toma un argumento y no devuelve nada.

Podemos implementarlo creando una clase que implemente el interfaz `Consumer` y luego creando un objeto de dicha clase. O también podemos hacer lo mismo utilizando una clase anónima:

```
listaPersonas.forEach(
    new Consumer<Persona>() {
        @Override
        public void accept(Persona p) {
            System.out.println(p.getNombre());
        }
    }
);
```

No es objeto del curso profundizar en las clases anónimas, sirva simplemente como recordatorio indicar que son una forma ágil de crear objetos que implementan interfaces.

En cualquier caso, como vemos el código no parece mejorar respecto al de la primera solución, más bien al contrario.

Tercera solución: método `foreach` y expresión `lambda`.

En esta solución utilizaremos el método `foreach` y, puesto que el argumento es un interfaz funcional, utilizaremos una expresión `lambda` en vez de la clase anónima:

```
listaPersonas.forEach(p -> System.out.println(p.getNombre()));
```

Vemos que, ahora sí, el código queda más compacto. En este caso “`p`” es el nombre que le damos al argumento del interfaz funcional; al igual que cuando definimos los argumentos de un método, podemos poner el nombre que queramos. Es decir, que sería equivalente poner lo siguiente:

```
listaPersonas.forEach(persona -> System.out.println(persona.getNombre()));
```

Ejemplo 2

Tomando como punto de partida la lista del ejemplo anterior, queremos ordenarla por orden alfabético del primer apellido y mostrar por pantalla los primeros apellidos ordenados:

```
listaPersonas.sort( (p1,p2) -> p1.getApellido1().compareTo(p2.getApellido1()));
listaPersonas.forEach(p -> System.out.println(p.getApellido1()));
```

En este ejemplo, primero ordenamos la lista y a continuación mostramos los apellidos. La primera expresión `lambda` está implementando el método [compare](#) del interfaz [Comparator](#). Obsérvese que este método tiene dos argumentos, por lo que la expresión `lambda` también debe tener dos parámetros de entrada.

Recordemos que el interfaz `Comparator` nos permite definir un *criterio* de ordenación, para luego utilizarlo dentro de una función de ordenación que es la que implementa el algoritmo de ordenación. Así pues, el `Comparator` define, para dos objetos de un determinado tipo, cuál se considera mayor que el otro, o si se consideran iguales. La función `sort` del interfaz `List()` implementará el algoritmo, y utilizará el `Comparator` para ordenar los elementos de dos en dos dentro de su algoritmo. En concreto el método `compare` del interfaz `Comparator` se define de la siguiente forma:

Devuelve un número entero negativo, cero o positivo si el primer argumento es menor, igual o mayor que el segundo, respectivamente.

En nuestro ejemplo, para dos personas, queremos ordenar por el orden alfabético del primer apellido. Para ello nos apoyamos en el método [compareTo](#) de la clase *String*, que precisamente nos devuelve un número negativo, cero o positivo al ser comparada con otra cadena de texto, según el orden alfabético.

Ejemplo 3

En caso de que queramos ejecutar más de una sentencia en la expresión, debemos utilizar llaves al principio y al final, y separar las sentencias por punto y coma:

```
listaPersonas.forEach(p -> {  
    System.out.println(p.getApellido1());  
    System.out.println();  
});
```

2. Los Streams y las expresiones Lambda

La utilidad de las expresiones lambda se puede apreciar cuando se utilizan en combinación con las funcionalidades ofrecidas por el paquete [java.util.stream](#). Este paquete nos permite realizar operaciones funcionales sobre colecciones de forma encadenada, tales como filtrado, ordenación, etc. En el lenguaje C# el equivalente sería LINQ (Language Integrated Query).

Un Stream es como una lista de objetos, pero debe entenderse como un flujo, es decir, que los elementos se van consumiendo a medida que se van necesitando. Hay que recalcar que no hay que confundir estos stream con los *InputStream* y *OutputStream* del paquete *java.io*, que sirven para entrada/salida, aunque tienen similitudes.

Veamos un ejemplo: partimos de la lista de personas del ejemplo anterior, y queremos mostrar por pantalla la lista de primeros apellidos de aquellas personas cuyo segundo apellido no es "HERRERA", ordenados alfabéticamente y sin duplicados. Utilizando streams junto con expresiones lambda podemos realizarlo en una única sentencia:

```
listaPersonas.stream()  
    .filter(p -> !p.apellido2.equals("HERRERA"))  
    .map(p -> p.apellido1)  
    .distinct()  
    .sorted()  
    .forEach(apellido -> System.out.println(apellido));
```

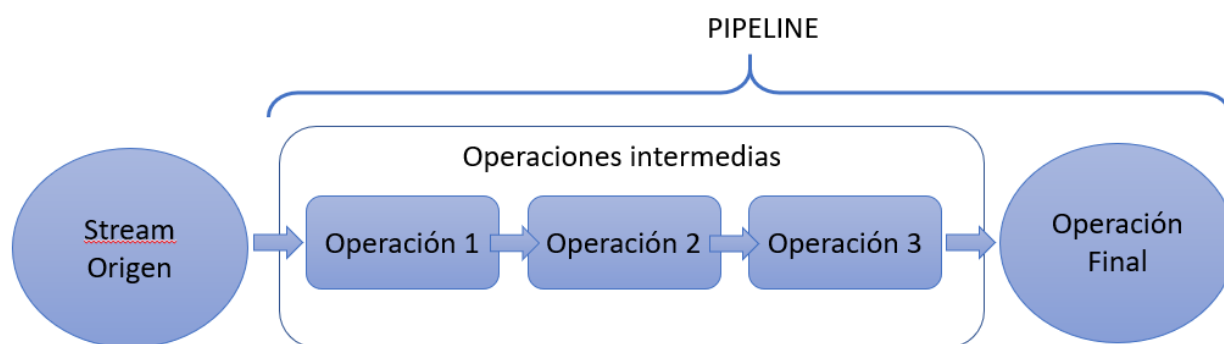
A continuación, explicamos en detalle la sentencia:

- En primer lugar el método [stream](#) del interfaz *List*, heredado del interfaz *Collection*, devuelve un objeto *Stream*, que es el que nos permite acceder a toda la funcionalidad.
- A continuación, el método [filter](#) nos devuelve un nuevo stream de objetos *Persona* pero conteniendo sólo los que cumplen la condición indicada como argumento, que es una expresión lambda que debe devolver un boolean. A la expresión lambda que devuelve un boolean se le llama "predicado".
- A continuación, el método [map](#) nos permite convertir la lista de objetos *Persona* en otra lista de objetos de otro tipo. Así pues, "extraemos" el apellido 1, pasando de una lista de objetos *Persona* a una lista de objetos *String* que contienen, para cada elemento, el valor del atributo *apellido1*.

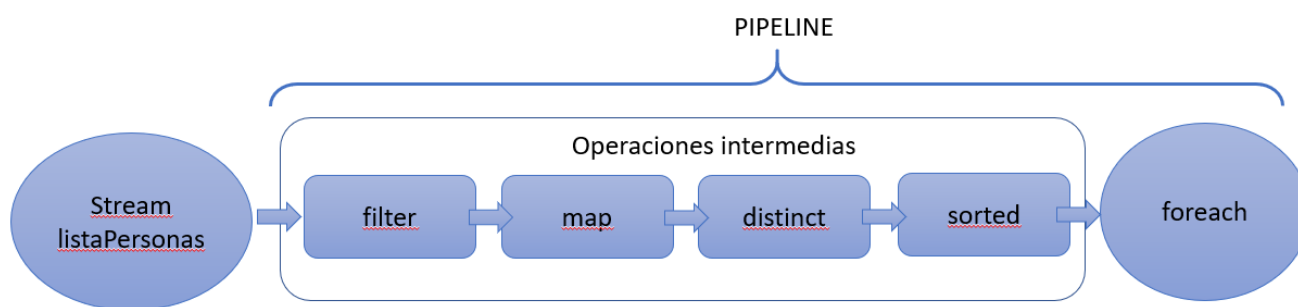
- El método [distinct](#) elimina los duplicados, de acuerdo al método equals de la clase de los objetos (en este caso la clase String).
- El método [sorted](#) ordena los elementos de acuerdo a su orden natural. Los elementos deben implementar el interfaz [Comparable](#). En este caso la clase String lo implementa y es el orden alfabético.
- Finalmente el método [foreach](#) nos permite ejecutar una expresión para cada elemento, como por ejemplo mostrarlo por pantalla.

Observamos el gran potencial que tiene el encadenamiento de streams, que permite reducir la cantidad de código necesario, mejorando la legibilidad y también el mantenimiento.

En los streams hay dos tipos de operaciones, *intermedias* y *finales*, que se van encadenando formando tuberías o *pipelines*. Un pipeline está formado por una fuente (source), seguida opcionalmente por una o varias operaciones intermedias (como filter, map, etc), y a continuación una operación final (como foreach). Las operaciones intermedias devuelven un nuevo stream. Estas operaciones intermedias no se ejecutan inmediatamente, es decir, el recorrido de todo el pipeline no empieza a ejecutarse hasta que se invoca la operación final.



En nuestro ejemplo sería:



A continuación, se muestran algunos ejemplos de operaciones intermedias:

- *filter*: permite filtrar los elementos que cumplen una condición (predicado).
- *map*: aplica una función a cada elemento.
- *flatMap*: similar a map pero permite conversiones de 1 elemento fuente a n elementos destino.
- *distinct*: elimina duplicados.
- *sorted()*: ordena por orden natural.
- *sorted(Comparator<? super T> comparator)*: ordena por el orden indicado.
- *peek*: ejecuta una acción para cada elemento. Útil para debug, por ejemplo:

```

listaPersonas.stream()
    .filter(p -> !p.apellido2.equals("HERRERA"))
    .map(p -> p.apellido1)
    .peek(ap -> System.out.println("apellido es: " + ap))
    .distinct()
    .sorted()
    .forEach(apellido -> System.out.println(apellido));

```

- *limit*(long n): devuelve los primeros n elementos del stream.
- *skip*(long n): descarta los primeros n elementos del stream. El uso conjunto de limit y skip permite por ejemplo paginar un conjunto de resultados: primero usaríamos skip para buscar el comienzo de la página, y luego limit(n) siendo n el tamaño de la página.

Ejemplos de operaciones finales:

- *foreach*: ejecuta una acción por cada elemento.
- *toArray*: devuelve un array.
- *reduce* y *collect*: aplican una operación de reducción que se pasa como argumento. Las operaciones de reducción son típicamente operaciones de agregado, como pueden ser suma, mínimo, máximo, media, concatenación, etc. La clase [Collectors](#) proporciona diversas utilidades para este fin.
- *min*: devuelve el mínimo.
- *max*: devuelve el máximo.
- *count*: devuelve el número de elementos.
- *anyMatch*: devuelve true si algún elemento cumple el predicado que se pasa como argumento.
- *allMatch*: devuelve true si todos los elementos cumplen el predicado que se pasa como argumento.
- *noneMatch*: devuelve true si ningún elemento cumple el predicado que se pasa como argumento.
- *findFirst*: devuelve el primer elemento.

Ejemplos:

```

//Obtener la persona que tiene el primer apellido más corto.
//Al método min debemos pasarle como argumento un Comparator, tal que para
//dos personas nos devuelva un número negativo si la primera persona tiene el //apellido
más corto (ver documentación del interfaz Comparator).
Persona p = listaPersonas.stream()
    .min( (p1,p2) -> p1.getApellido1().length() - p2.getApellido1().length()).get();

```

```

System.out.println("Persona con el apellido más corto: " + p);

```

```

//número de personas que se apellidan FERNANDEZ:
long c = listaPersonas.stream()
    .filter(persona -> persona.getApellido1().equals("FERNANDEZ"))
    .count();

```

```

System.out.println("Número de personas que se apellidan FERNANDEZ: " + c);

```

```

//agrupar personas por primer apellido:
//nos devuelve un Map, la clave es el primer apellido, el valor es
//la lista de personas con ese primer apellido.
Map<String, List<Persona>> agrupadas = listaPersonas.stream()
    .collect(Collectors.groupingBy(Persona::getApellido1));

```

```

System.out.println("Personas agrupadas por primer apellido:");

```

```

//entrySet es un método de Map que nos devuelve todas las entry del Map

```

```
//Cada entry es una pareja clave - valor.
agrupadas.entrySet().forEach(entry -> {
    String primerApellido = entry.getKey();
    System.out.println(primerApellido + ":");

    List<Persona> listPersonas = entry.getValue();
    listPersonas.forEach(persona -> System.out.println(" " + persona.toString()));
});

//Contar las personas agrupando por la inicial del primer apellido:
Map<String, Long> cuentaPorIniciales = listaPersonas.stream()
    .collect(Collectors.groupingBy(
        persona -> persona.getApellido1().substring(0, 1), //agrupación
        Collectors.counting() //reducción
    ));

cuentaPorIniciales.entrySet().forEach(entry -> {
    String inicial = entry.getKey();
    Long cuenta = entry.getValue();
    System.out.println("Hay " + cuenta + " persona(s) cuyo primer apellido empieza por "
        + inicial);
});
```

3. Ejercicio resuelto

Se está desarrollando una aplicación que mostrará un listado de clientes que se obtienen de una fuente externa, como podría ser un servicio web. En el ejercicio para simplificar se obtendrá el listado de un método que simulará la fuente externa.

Los clientes tendrán los siguientes atributos: id, nombre, apellidos.

Se desea que el usuario final pueda consultar el listado en forma de tabla paginada, es decir, si el tamaño de la página es por ejemplo 10 y visualizo la segunda página, se deben mostrar los clientes que ocupan las posiciones 11 al 20 en el listado. El usuario debe poder ordenar por cualquier columna (opcional).

Se nos encarga implementar un método que devuelva el listado de clientes correspondiente a la página seleccionada. El método tendrá como entradas:

- La lista completa de clientes
- El número de página
- El tamaño de la página (nº de clientes en cada página del listado).
- El campo por el que se desea ordenar (opcional)

Se debe implementar este método utilizando *streams* y expresiones lambda (métodos *skip*, *limit*, *sorted*).

El alumno deberá intentar resolver el ejercicio antes de ver la solución. Se facilita el código del esqueleto de la clase (disponible en la plataforma).